

Reporte Técnico: Dashboard de Análisis de Ventas con Next.js y PostgreSQL

Autor: Juan Manuel Camacho
Fecha: 6 de febrero de 2026
Materia: Aplicaciones Web Orientadas a Servicios (AWOS)

1. Introducción

Este documento presenta el análisis y desarrollo de una aplicación web orientada a servicios para la visualización de reportes empresariales mediante vistas SQL en PostgreSQL. El proyecto implementa un dashboard interactivo utilizando Next.js 14 con TypeScript, cumpliendo con requisitos de seguridad, filtrado de datos y separación de responsabilidades.

2. Arquitectura del Sistema

2.1 Stack Tecnológico

El sistema se compone de las siguientes tecnologías:

- **Frontend:** Next.js 14 (App Router) con TypeScript y Tailwind CSS
- **Backend:** API Routes de Next.js para endpoints RESTful
- **Base de Datos:** PostgreSQL 16 Alpine
- **Ciente de BD:** node-postgres (pg) con pool de conexiones
- **Orquestación:** Docker Compose para servicios multi-contenedor
- **Seguridad:** Usuario de aplicación con permisos restringidos

Justificación técnica: Se eligió Next.js 14 por su capacidad de renderizado del lado del servidor (SSR) y rutas de API integradas, lo que permite una arquitectura monolítica simplificada pero escalable. PostgreSQL fue seleccionado por su robustez en el manejo de vistas complejas y características avanzadas de seguridad a nivel de base de datos[1].

2.2 Arquitectura de Seguridad

La aplicación implementa el principio de mínimo privilegio mediante:

1. **Usuario de aplicación (app_user):** Usuario PostgreSQL dedicado sin privilegios de superusuario
2. **Vistas con Security Barrier:** Todas las vistas utilizan `WITH (security_barrier = true)` para prevenir ataques de timing[2]
3. **Propietario de vistas:** Todas las vistas son propiedad del usuario `postgres` con `ALTER VIEW ... OWNER TO postgres`
4. **Permisos restrictivos:** El usuario `app_user` solo tiene permiso `SELECT` sobre las vistas, sin acceso directo a tablas base

```
-- Ejemplo de configuración de seguridad
CREATE OR REPLACE VIEW vw_sales_daily
WITH (security_barrier = true) AS
SELECT ...;

ALTER VIEW vw_sales_daily OWNER TO postgres;

GRANT SELECT ON vw_sales_daily TO app_user;
```

Justificación: Esta arquitectura previene que la aplicación realice operaciones de escritura no autorizadas y protege los datos sensibles mediante la capa de abstracción de las vistas[3].

3. Estructura de la Base de Datos

3.1 Esquema de Tablas

El modelo de datos implementa un esquema normalizado con 10 tablas principales:

- **categories:** Categorías de productos
- **suppliers:** Proveedores
- **products:** Catálogo de productos con stock
- **customers:** Base de clientes
- **orders:** Órdenes de compra con estado y canal
- **order_items:** Detalle de productos por orden
- **payments:** Métodos y montos de pago
- **product_costs:** Costos históricos de productos
- **inventory_movements:** Movimientos de entrada/salida
- **order_status_history:** Historial de cambios de estado

Volumen de datos:

- 100 clientes
- 300 órdenes (agosto 2024 - febrero 2026)
- 13 productos distribuidos en 5 categorías
- 4 proveedores

3.2 Vistas Analíticas Implementadas

El sistema incluye 12 vistas SQL diseñadas para diferentes análisis de negocio:

Vista 1: Ventas Diarias (**vw_sales_daily**)

Propósito: Agregación de ventas por fecha con métricas de tickets promedio.

```
CREATE OR REPLACE VIEW vw_sales_daily
WITH (security_barrier = true) AS
SELECT
```

```
DATE(o.created_at) AS sale_date,
SUM(oi.qty * oi.unit_price) AS total_ventas,
COUNT(DISTINCT o.id) AS tickets,
ROUND(SUM(oi.qty * oi.unit_price) / NULLIF(COUNT(DISTINCT o.id), 0), 2) AS
ticket_promedio
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
WHERE o.status = 'COMPLETED'
GROUP BY DATE(o.created_at);
```

Justificación: Esta vista permite análisis de tendencias temporales y patrones de ventas diarios. El uso de `NULLIF` previene divisiones por cero, mejorando la robustez del código[4].

Filtros implementados:

- `date_from`: Fecha inicial del rango
- `date_to`: Fecha final del rango

[INSERTAR CAPTURA DE PANTALLA: Dashboard - Sección Ventas Diarias]

Vista 2: Top Productos Rankeados (`vw_top_products_ranked`)

Propósito: Ranking de productos por unidades vendidas y revenue.

```
CREATE OR REPLACE VIEW vw_top_products_ranked
WITH (security_barrier = true) AS
SELECT
    p.id AS product_id,
    p.name AS product_name,
    SUM(oi.qty) AS unidades_vendidas,
    SUM(oi.qty * oi.unit_price) AS revenue,
    RANK() OVER (ORDER BY SUM(oi.qty * oi.unit_price) DESC) AS ranking
FROM products p
JOIN order_items oi ON oi.product_id = p.id
JOIN orders o ON o.id = oi.order_id
WHERE o.status = 'COMPLETED'
GROUP BY p.id, p.name;
```

Justificación: Utiliza funciones de ventana (`RANK()`) para asignar posiciones sin gaps en caso de empates, ideal para análisis competitivo de productos[5].

Funcionalidades implementadas:

- **Búsqueda:** Filtro por nombre de producto usando `ILIKE`
- **Paginación:** Soporte de `page` y `limit` con metadata de navegación

[INSERTAR CAPTURA DE PANTALLA: Página de Productos con búsqueda y paginación]

Vista 3: Riesgo de Inventario (vw_inventory_risk)

Propósito: Identificación de categorías con productos en stock bajo.

```
CREATE OR REPLACE VIEW vw_inventory_risk
WITH (security_barrier = true) AS
SELECT
  c.id AS category_id,
  c.name AS category_name,
  COUNT(p.id) AS productos_totales,
  SUM(CASE WHEN p.stock < 10 THEN 1 ELSE 0 END) AS productos_en_riesgo,
  ROUND(
    SUM(CASE WHEN p.stock < 10 THEN 1 ELSE 0 END)::numeric
    / NULLIF(COUNT(p.id), 0) * 100,
    2
  ) AS porcentaje_riesgo
FROM products p
JOIN categories c ON c.id = p.category_id
GROUP BY c.id, c.name
HAVING COUNT(p.id) > 0;
```

Justificación: La agregación condicional con **CASE** permite calcular porcentajes de riesgo de manera eficiente en una sola consulta, evitando subconsultas adicionales[6].

Filtros implementados:

- **Whitelist de categorías:** Validación del lado del servidor para prevenir inyección SQL

```
const VALID_CATEGORIES = ['Café', 'Bebidas frías', 'Panadería', 'Postres',
  'Snacks'];
if (category && !VALID_CATEGORIES.includes(category)) {
  return NextResponse.json({ error: 'Invalid category' }, { status: 400 });
}
```

[INSERTAR CAPTURA DE PANTALLA: Dashboard - Inventario en Riesgo con filtros de categoría]

Vista 4: Valor del Cliente (vw_customer_value)

Propósito: Análisis de valor de vida del cliente (CLV básico).

```
CREATE OR REPLACE VIEW vw_customer_value
WITH (security_barrier = true) AS
SELECT
  c.id AS customer_id,
  c.name AS customer_name,
  COUNT(DISTINCT o.id) AS num_ordenes,
  SUM(oi.qty * oi.unit_price) AS total_gastado,
```

```
ROUND(SUM(oi.qty * oi.unit_price) / NULLIF(COUNT(DISTINCT o.id), 0), 2) AS
gasto_promedio
FROM customers c
JOIN orders o ON o.customer_id = c.id
JOIN order_items oi ON oi.order_id = o.id
WHERE o.status = 'COMPLETED'
GROUP BY c.id, c.name;
```

Justificación: Permite segmentación de clientes por valor, facilitando estrategias de retención y marketing dirigido[7].

Funcionalidad:

- **Paginación:** Lista completa de clientes con navegación por páginas

[INSERTAR CAPTURA DE PANTALLA: Página de Clientes con métricas de valor]

Vista 5: Mezcla de Pagos (vw_payment_mix)

Propósito: Distribución porcentual de métodos de pago.

```
CREATE OR REPLACE VIEW vw_payment_mix
WITH (security_barrier = true) AS
SELECT
    method,
    SUM(paid_amount) AS total_pagado,
    ROUND(
        SUM(paid_amount) / NULLIF(SUM(SUM(paid_amount)) OVER (), 0) * 100,
        2
    ) AS porcentaje
FROM payments
GROUP BY method;
```

Justificación: Utiliza funciones de ventana para calcular porcentajes del total sin necesidad de subconsultas o JOIN con agregaciones previas[8].

[INSERTAR CAPTURA DE PANTALLA: Dashboard - Gráfico de distribución de métodos de pago]

Vista 6: Rentabilidad por Producto (vw_product_profitability)

Propósito: Cálculo de margen de ganancia usando CTE.

```
CREATE OR REPLACE VIEW vw_product_profitability
WITH (security_barrier = true) AS
WITH latest_cost AS (
    SELECT pc.product_id, pc.cost
    FROM product_costs pc
```

```

        JOIN (
            SELECT product_id, MAX(effective_from) AS max_date
            FROM product_costs
            GROUP BY product_id
        ) lc ON lc.product_id = pc.product_id
        AND lc.max_date = pc.effective_from
    )
SELECT
    p.id AS product_id,
    p.name AS product_name,
    SUM(oi.qty * oi.unit_price) AS revenue,
    SUM(oi.qty * lc.cost) AS total_cost,
    SUM(oi.qty * oi.unit_price) - SUM(oi.qty * lc.cost) AS profit
FROM products p
JOIN order_items oi ON oi.product_id = p.id
JOIN orders o ON o.id = oi.order_id
JOIN latest_cost lc ON lc.product_id = p.id
WHERE o.status = 'COMPLETED'
GROUP BY p.id, p.name;

```

Justificación: El uso de Common Table Expressions (CTE) mejora la legibilidad y permite consultas jerárquicas para obtener el costo más reciente de cada producto[9].

Vistas Adicionales

Vista 7: Rotación de Inventario (**vw_inventory_turnover**)

- Analiza movimientos de entrada y salida por producto
- Identifica productos con baja rotación

Vista 8: Ciclo de Vida de Pedidos (**vw_order_lifecycle**)

- Mide tiempos de procesamiento usando **AGE()**
- Identifica cuellos de botella operacionales

Vistas 9-12: Métricas del Dashboard

- **vw_today_sales**: Ventas del día actual
- **vw_today_orders**: Órdenes del día actual
- **vw_low_stock_count**: Conteo de productos críticos
- **vw_products_list**: Catálogo completo con JOIN de categorías

[INSERTAR CAPTURA DE PANTALLA: Dashboard principal con todas las métricas]

4. Implementación de APIs

4.1 Estructura de Endpoints

Todos los endpoints siguen el patrón RESTful con respuestas JSON estructuradas:

```
// Estructura de respuesta exitosa
{
  "success": true,
  "data": { /* datos de la vista */ },
  "pagination": { /* metadata de paginación */ }
}

// Estructura de respuesta de error
{
  "success": false,
  "error": "Mensaje de error descriptivo"
}
```

4.2 Prevención de Inyección SQL

Todas las consultas utilizan parametrización:

```
// ☒ CORRECTO - Consulta parametrizada
const query = 'SELECT * FROM vw_sales_daily WHERE sale_date >= $1 AND sale_date <= $2';
const result = await db.query(query, [dateFrom, dateTo]);

// ☐ INCORRECTO - Concatenación de strings (vulnerable)
const query = `SELECT * FROM vw_sales_daily WHERE sale_date >= '${dateFrom}'`;
```

Justificación: Las consultas parametrizadas previenen inyección SQL al separar lógica de datos, siendo la defensa primaria recomendada por OWASP[10].

4.3 Paginación del Lado del Servidor

Implementación con metadata completa:

```
const page = parseInt(searchParams.get('page') || '1');
const limit = parseInt(searchParams.get('limit') || '10');
const offset = (page - 1) * limit;

const countResult = await db.query('SELECT COUNT(*) as total FROM vw_customer_value');
const totalRecords = parseInt(countResult.rows[0].total);
const totalPages = Math.ceil(totalRecords / limit);

const dataResult = await db.query(
  'SELECT * FROM vw_customer_value ORDER BY total_gastado DESC LIMIT $1 OFFSET $2',
  [limit, offset]
);

return NextResponse.json({
```

```

    success: true,
    data: dataResult.rows,
    pagination: {
      currentPage: page,
      totalPages,
      totalRecords,
      recordsPerPage: limit,
      hasNextPage: page < totalPages,
      hasPrevPage: page > 1
    }
  });

```

Justificación: La paginación del lado del servidor reduce la carga de red y mejora el rendimiento en conjuntos de datos grandes, siguiendo principios de REST[11].

5. Optimizaciones de Base de Datos

5.1 Índices Implementados

```

-- Índices para optimización de JOIN
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_order_items_product_id ON order_items(product_id);

-- Índices para filtrado de fecha
CREATE INDEX idx_orders_created_at ON orders(created_at);
CREATE INDEX idx_orders_created_at_status ON orders(created_at, status);

-- Índices para búsqueda
CREATE INDEX idx_products_stock ON products(stock);
CREATE INDEX idx_products_active ON products(active);

```

Justificación: Los índices B-tree mejoran el rendimiento de JOIN, WHERE y ORDER BY en órdenes de magnitud $O(\log n)$ vs $O(n)$ [12].

5.2 Pool de Conexiones

```

const pool = new Pool({
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT || '5432'),
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  max: 20, // Máximo de conexiones en el pool
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});

```


Justificación: El pool de conexiones reduce overhead de creación/destrucción de conexiones TCP, mejorando throughput en aplicaciones con alta concurrencia[13].

6. Capturas de Pantalla

6.1 Dashboard Principal

[INSERTAR CAPTURA: Vista general del dashboard mostrando métricas del día, ventas recientes, top productos y alertas de inventario]

6.2 Página de Ventas

[INSERTAR CAPTURA: Vista de ventas diarias con filtros de fecha aplicados y tabla de resultados]

6.3 Página de Productos

[INSERTAR CAPTURA: Lista de productos rankeados con búsqueda activa y controles de paginación]

6.4 Página de Inventario

[INSERTAR CAPTURA: Dashboard de inventario con categorías filtradas y alertas de stock bajo]

6.5 Página de Clientes

[INSERTAR CAPTURA: Lista de clientes ordenados por valor total gastado con paginación]

6.6 Verificación de Seguridad

[INSERTAR CAPTURA: Terminal mostrando permisos de app_user limitados solo a SELECT en vistas]

7. Conclusiones

7.1 Logros Técnicos

- Arquitectura de Seguridad Robusta:** La implementación de vistas con security barrier y usuario de aplicación con permisos mínimos proporciona una capa de protección efectiva contra accesos no autorizados y modificaciones maliciosas de datos.
- Separación de Responsabilidades:** La arquitectura de tres capas (presentación, API, datos) permite mantenibilidad y escalabilidad. Las vistas SQL encapsulan la lógica de negocio en la capa de datos, reduciendo código duplicado en la aplicación.
- Rendimiento Optimizado:** La combinación de índices estratégicos, pool de conexiones y paginación del lado del servidor permite manejar volúmenes crecientes de datos sin degradación significativa de rendimiento.
- Prevención de Vulnerabilidades:** El uso consistente de consultas parametrizadas y validación de whitelist elimina vectores comunes de ataque como inyección SQL y manipulación de parámetros.

7.2 Decisiones de Diseño Destacadas

- **Next.js App Router:** Aprovecha renderizado híbrido (SSR/CSR) para optimizar tiempo de carga inicial y SEO
- **PostgreSQL sobre MySQL:** Mayor soporte para vistas complejas, CTEs y funciones de ventana
- **Docker Compose:** Facilita despliegue reproducible y aislamiento de servicios
- **TypeScript:** Prevención de errores en tiempo de compilación, mejora autocompletado

7.3 Escalabilidad Futura

El sistema está preparado para:

- Implementación de caché con Redis para consultas frecuentes
- Sharding horizontal de PostgreSQL para datasets masivos
- Migración a arquitectura de microservicios si se requiere mayor granularidad
- Implementación de autenticación JWT y autorización basada en roles

7.4 Aprendizajes Clave

1. Las vistas SQL bien diseñadas actúan como contratos de API, proporcionando estabilidad a la interfaz de datos
2. La seguridad debe implementarse en múltiples capas (aplicación, red, base de datos)
3. El análisis de datos requiere equilibrio entre completitud de información y rendimiento de consultas
4. La documentación clara de decisiones arquitectónicas facilita mantenimiento a largo plazo

8. Fuentes de Consulta

[1] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation - Views and Rules*. <https://www.postgresql.org/docs/16/rules-views.html>

[2] PostgreSQL Global Development Group. (2024). *PostgreSQL Security - Row Security Policies*. <https://www.postgresql.org/docs/16/ddl-rowsecurity.html>

[3] OWASP Foundation. (2023). *OWASP Top 10 - Broken Access Control*. https://owasp.org/Top10/A01_2021-Broken_Access_Control/

[4] PostgreSQL Global Development Group. (2024). *PostgreSQL Functions and Operators - Conditional Expressions*. <https://www.postgresql.org/docs/16/functions-conditional.html>

[5] PostgreSQL Global Development Group. (2024). *Window Functions Tutorial*. <https://www.postgresql.org/docs/16/tutorial-window.html>

[6] PostgreSQL Global Development Group. (2024). *Aggregate Functions*. <https://www.postgresql.org/docs/16/functions-aggregate.html>

[7] Kumar, V., & Reinartz, W. (2018). *Customer Relationship Management: Concept, Strategy, and Tools* (3rd ed.). Springer.

[8] PostgreSQL Global Development Group. (2024). *Window Function Processing*. <https://www.postgresql.org/docs/16/queries-table-expressions.html#QUERIES-WINDOW>

[9] PostgreSQL Global Development Group. (2024). *Common Table Expressions*.

<https://www.postgresql.org/docs/16/queries-with.html>

[10] OWASP Foundation. (2023). *SQL Injection Prevention Cheat Sheet*.

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[11] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Doctoral dissertation, University of California, Irvine].

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

[12] Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.

[13] node-postgres contributors. (2024). *Connection Pooling*. <https://node-postgres.com/features/pooling>

[14] Vercel. (2024). *Next.js 14 Documentation - App Router*. <https://nextjs.org/docs/app>

[15] Mozilla Developer Network. (2024). *HTTP Status Codes*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

Apéndice A: Comandos de Docker

```
# Iniciar servicios
docker-compose up -d

# Verificar estado
docker-compose ps

# Ver logs
docker-compose logs web
docker-compose logs db

# Detener y eliminar volúmenes
docker-compose down -v

# Reconstruir sin caché
docker-compose build --no-cache
```

Apéndice B: Variables de Entorno

```
# Configuración de Base de Datos
POSTGRES_DB=awos
POSTGRES_USER=postgres
POSTGRES_PASSWORD=anona29050XD

# Configuración de Aplicación
DATABASE_URL=postgresql://app_user:app_secure_password_2024@db:5432/awos
NEXT_PUBLIC_API_BASE_URL=http://localhost:3001
```

Apéndice C: Estructura de Archivos de Base de Datos

DB/		
—	01_schema.sql	# Definición de tablas
—	02_seed.sql	# Datos de prueba (100 clientes, 300 órdenes)
—	03_reports_vw.sql	# Vistas analíticas (12 vistas)
—	04_indexes.sql	# Índices de optimización
—	05_roles.sql	# Usuario app_user y permisos

Fin del Reporte

Este documento fue elaborado como parte del proyecto de Aplicaciones Web Orientadas a Servicios, implementando principios de arquitectura limpia, seguridad por diseño y mejores prácticas de desarrollo web moderno.